

LENCORD

DOCUMENTACIÓN DE INGENIERÍA & OPERACIONES

MANUAL TÉCNICO DE PUESTA EN MARCHA Y APAGADO

Guía completa de instalación, ejecución en entornos de desarrollo y producción, gestión de puertos y detención segura de procesos.

Plataforma: Lencord SAS | Next.js 16 • React 19 • TypeScript

Versión: 1.0.0 | Entorno: Standalone / Hybrid

1. Introducción y Arquitectura del Sistema

Lencord es una plataforma tecnológica de financiamiento colectivo peer-to-peer (P2P) orientada al ecosistema de pequeñas y medianas empresas (PyMEs) de la República Argentina. Permite vincular solicitudes crediticias corporativas con inversores particulares e institucionales mediante subastas de fondeo bajo la modalidad "todo o nada", instrumentadas jurídicamente mediante pagarés digitales y contratos de mutuo electrónico.

1.1. Stack Tecnológico

El núcleo de la aplicación fue desarrollado bajo los estándares más modernos del ecosistema web:

- **Framework Principal:** Next.js 16 (arquitectura App Router con Server Components y Server Actions).
- **Librería de Interfaz:** React 19 con componentes fuertemente tipados.
- **Lenguaje:** TypeScript 5 configurado en modo estricto (`strict: true`).
- **Motor de Pruebas:** Vitest 5 integrado con Testing Library y Happy-DOM para tests unitarios y de integración.
- **Estilizado y Diseño:** Vanilla CSS Modules con variables semánticas centralizadas y soporte de diseño responsivo.
- **Capa de Persistencia y Autenticación:** Supabase SSR (PostgreSQL) con patrón adaptador de *Mock Services* en memoria para entornos desacoplados.

1.2. Patrón de Servicios Híbrido (Service Provider Pattern)

La plataforma cuenta con un sistema de inyección de dependencias (`ServiceProvider`) que permite operar al 100% de manera autónoma sin bases de datos externas activas utilizando servicios mock en memoria precargados con datos semilla del mercado argentino (`NEXT_PUBLIC_USE MOCKS=true`), o conectarse a una instancia real de PostgreSQL / Supabase (`NEXT_PUBLIC_USE MOCKS=false`).

Modo por Defecto: El sistema está configurado para operar de forma inmediata en modo Mock sin requerir configuración previa de bases de datos externas, permitiendo simular solicitudes de PyMEs, scoring BCRA, posturas de inversores y firmas de pagarés electrónicos.

2. Requisitos Previos del Sistema

Antes de levantar la aplicación, asegúrese de contar con las siguientes herramientas instaladas en su equipo host:

Componente	Versión Requerida	Verificación en Terminal
Node.js	v20.x o superior (Probado en v24.x LTS)	<code>node -v</code>
Gestor de paquetes npm	v10.x o superior (incluido con Node.js)	<code>npm -v</code>
Sistema Operativo	Windows 10/11, Linux (Ubuntu/Debian) o macOS	Terminal PowerShell, Bash o Zsh
Navegador Web	Edge, Chrome, Firefox o Safari actualizados	Resolución estándar de escritorio y móvil

3. Cómo Hacer Andar la Página (Puesta en Marcha)

Para poner en funcionamiento el servidor local y acceder a la plataforma, siga minuciosamente los siguientes pasos desde la terminal de comandos de su sistema operativo:

Paso 3.1: Ubicación en el Directorio del Proyecto

Abra una consola de comandos (PowerShell o CMD en Windows, Terminal en Linux/macOS) y navegue a la carpeta raíz del proyecto:

```
cd c:\Users\SYC\Desktop\lencord
```

Paso 3.2: Instalación de Dependencias

Si es la primera vez que ejecuta el proyecto o se han clonado cambios recientes, instale los paquetes de Node definidos en `package.json`:

```
npm install
```

Regla de oro de dependencias: No instale paquetes adicionales mediante `npm i <libreria>` sin previa autorización técnica del equipo de arquitectura, ya que el proyecto mantiene un control estricto de dependencias en `package.json`.

Paso 3.3: Ejecución en Modo Desarrollo (Recomendado)

El modo desarrollo activa el compilador incremental con recarga rápida en caliente (Hot Module Replacement / Fast Refresh). Ejecute el siguiente comando:

```
npm run dev
```

Al iniciar, observará un mensaje similar al siguiente en la terminal:

```
▲ Next.js 16.3.6
- Local:      http://localhost:3000
- Environments: .env.local

✓ Starting...
✓ Ready in 1200ms
```

Abra su navegador web favorito e ingrese a la siguiente dirección URL:

```
http://localhost:3000
```

¡Plataforma Operativa! Ya puede navegar libremente por la Landing Page, simular créditos, cargar solicitudes en `/solicitar`, explorar oportunidades en `/marketplace`, gestionar inversiones en `/dashboard` y acceder al backoffice en `/admin`.

Paso 3.4: Ejecución en Puerto Alternativo (en caso de puerto 3000 ocupado)

Si el puerto estándar `3000` estuviera siendo utilizado por otra aplicación, Next.js intentará automáticamente saltar al `3001`. También puede forzar un puerto específico ejecutando:

```
npx next dev -p 3005
```

Paso 3.5: Compilación y Ejecución en Modo Producción

Para verificar el rendimiento real de producción o validar que el código no presente errores de tipado o empaquetado, ejecute:

1 Compilar el bundle optimizado:

```
npm run build
```

Este paso genera los artefactos en la carpeta `.next/` e inspecciona los tipos TypeScript.

2 Iniciar el servidor productivo de alto rendimiento:

```
npm run start
```

Paso 3.6: Verificación de la Suite de Pruebas Automatizadas

Para comprobar la integridad de todos los módulos y componentes del sistema antes de operar:

```
npm run test
```

Para ejecutar una prueba específica de un componente (ej. Pagaré Digital):

```
npx vitest run tests/components/PromissoryNoteModal.test.tsx
```

4. Cómo Apagar y Detener la Página

El apagado de la plataforma debe realizarse de forma ordenada para liberar los puertos de red y asegurar que no queden procesos residuales en memoria consumiendo recursos del procesador.

4.1. Método Estándar Interactivo (Recomendado)

Si tiene la terminal abierta donde se inició el comando `npm run dev` o `npm run start`:

- 1 Haga clic sobre la ventana de la terminal activa.
- 2 Presione la combinación de teclas: `Ctrl + C` (o `Cmd + C` en macOS).
- 3 Si la consola de Windows consulta: `¿Desea terminar el trabajo por lotes (S/N)?`, escriba **S** y presione **Enter**.

El proceso de Node.js finalizará de inmediato y el puerto `3000` quedará liberado.

4.2. Método Forzado / Proceso en Segundo Plano (Windows PowerShell)

Si la terminal fue cerrada de forma abrupta o el servidor continúa corriendo en segundo plano impidiendo volver a levantarlo, utilice los siguientes comandos:

Opción A: Identificar PID y Terminar Proceso

1. Busque qué identificador de proceso (PID) está ocupando el puerto 3000:

```
netstat -ano | findstr :3000
```

Observará una línea como la siguiente (el último número de la derecha es el PID, por ejemplo `14520`):

```
TCP    0.0.0.0:3000      0.0.0.0:0        LISTENING        14520
```

2. Termine de forma forzada el proceso indicando dicho número:

```
taskkill /PID 14520 /F
```

Opción B: Detención Automática en una sola línea (PowerShell)

Ejecute directamente el siguiente comando en PowerShell para liberar el puerto 3000 sin buscar el PID manualmente:

```
Get-Process -Id (Get-NetTCPConnection -LocalPort 3000).OwningProcess -ErrorAction SilentlyContinue | Stop-Process -Force
```

4.3. Método en Entornos Linux / macOS

En sistemas operativos basados en Unix, ejecute:

```
# Buscar y matar el proceso en el puerto 3000
lsof -ti :3000 | xargs kill -9
```

5. Configuración de Variables de Entorno

El comportamiento del backend se configura a través del archivo `.env.local` ubicado en la raíz del proyecto.

Variable	Valores Posibles	Descripción y Propósito
<code>NEXT_PUBLIC_USE MOCKS</code>	<code>true</code> / <code>false</code>	Si es <code>true</code> , activa el almacenamiento en memoria con datos semilla. Si es <code>false</code> , requiere Supabase.
<code>NEXT_PUBLIC_SUPABASE_URL</code>	<code>https://xyz.supabase.co</code>	URL del proyecto Supabase en la nube o local (requerido si <code>USE MOCKS=false</code>).
<code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code>	Cadena JWT pública	Clave anónima para consultas del cliente web bajo políticas Row Level Security (RLS).
<code>SUPABASE_SERVICE_ROLE_KEY</code>	Cadena JWT privada	Clave administrativa restringida para Server Actions, Cron Jobs y Webhooks.
<code>PAYMENT_PROVIDER</code>	<code>mock</code> , <code>bind</code> , <code>pomelo</code>	Adaptador de pagos activo para retención, desembolsos y cobros automáticos.
<code>CRON_SECRET</code>	Clave alfanumérica	Token secreto de autorización para invocar el endpoint <code>/api/cron/check-deadlines</code> .

6. Resolución de Problemas Frecuentes (Troubleshooting)

Error: EADDRINUSE :::3000 Causa: Otra instancia previa de Node sigue escuchando en el puerto 3000. Solución: Ejecute el comando de detención forzada de la sección 4.2 o inicie con <code>npx next dev -p 3001</code>.	Caché corrupta o desincronizada Causa: Inconsistencias en el directorio temporal <code>.next</code> tras cambios de ramas. Solución: Borre la carpeta de compilación: <code>Remove-Item -Recurse -Force .next</code> y vuelva a levantar.
--	---